

SHIELD Architecture Series — Volume IV

Engineering Execution Blueprint

From Architecture to Working Software

This volume converts the SHIELD architecture into a structured, evidence-driven engineering program. It defines the MVP, the component workstreams, the repository structure, the schema contracts, the build sequence, and the acceptance criteria required to produce the first working Translation Engine and Composition Engine prototype.

CURRENT WORKING FRAMEWORK V0.2

ENGINEERING DRAFT

Document Objective

Volume IV serves as the bridge between the constitutional and operational architecture established in earlier volumes and the practical engineering program required to build, validate, and iterate on the first working system. This is not a design manifesto or a research proposal – it is a disciplined engineering plan oriented toward producing evidence of the architecture's validity through working software.

The prior volumes established the foundations on which this engineering program rests. Volume I established the constitutional foundation – the principles, values, and structural logic that define SHIELD as a system. Volume II defined the operational architecture, specifying the Translation Architecture and the Composition Architecture that give the system its functional shape. Volume III codified the human interaction doctrine – how human intelligence, judgment, and oversight integrate into every layer of the system.

Volume IV answers the next and most concrete question: how does the system begin to be built? It defines the first engineering target, the MVP boundaries, the component workstreams, the data flow, the schema contracts, the security and governance requirements, and the 90-day execution plan that transforms architecture into working software.

Volume I

Constitutional foundation –
principles, values, structural logic

Volume II

Operational architecture –
Translation Engine, Composition
Engine

Volume III

Human interaction doctrine –
oversight, judgment, collaboration

Volume IV

Engineering execution – build plan,
MVP, workstreams, roadmap

Audience

This document is written for the engineers, architects, and technical decision-makers who will design, build, validate, and deploy the SHIELD system. The intended audience includes software engineers, AI engineers, technical founders, engineering managers, university research engineers, systems architects, DevOps engineers, product architects, implementation partners, and technical investors.

This Document Is

- An engineering execution plan
- An MVP architecture specification
- A workstream blueprint
- A technical delivery roadmap
- An implementation planning document

This Document Is Not

- A pitch deck
- A business plan
- A design doctrine
- A research proposal
- A marketing document

Engineering Style and Standards

The engineering documentation style for Volume IV follows the discipline and precision characteristic of Stripe engineering architecture, AWS implementation guides, GitHub engineering handbooks, Apple platform engineering documentation, and MIT systems engineering briefs. These references establish the standard: precise, practical, technical, sequenced, disciplined, and evidence-driven.

Every specification in this volume is grounded in a real artifact, a defined schema contract, a testable acceptance criterion, or a versioned output. Speculative language, vague architectural assertions, and motivational framing have no place in this document. The system either works or it does not, and the evidence is the arbiter.

Precise

Every component has a defined input, output, and acceptance criterion. No ambiguous specifications.

Practical

Engineering decisions are grounded in real artifacts, real connectors, and real workflows.

Sequenced

Build phases are ordered by dependency. Nothing is built before its foundation is validated.

Evidence-Driven

Every case study becomes regression evidence. Architecture claims require working software proof.

Visual and Documentation Direction

Volume IV uses a dark premium visual language with electric teal accents and structured technical diagrams to support precision communication. System architecture diagrams, repository layouts, sprint maps, engineering flowcharts, data pipeline diagrams, API diagrams, schema diagrams, and component dependency maps are primary visual tools in this volume.



Architecture Diagrams

System component maps, data flow diagrams, and dependency graphs that show how the pipeline operates end to end.



Repository Structure

Annotated folder trees and module definitions that define the engineering workspace from day one.



Milestone Timelines

30/60/90-day delivery plans with clear phase gates and acceptance criteria for each phase.



Schema Contracts

Versioned JSON schema definitions that serve as the contract between architecture and implementation.

SHIELD Architecture Series — Volume IV

Engineering Execution Blueprint

From Architecture to Working Software

CURRENT WORKING FRAMEWORK V0.2

ENGINEERING DRAFT

Purpose of Volume IV

Volume IV serves one purpose: to convert the SHIELD architecture into a concrete, executable build plan. The prior three volumes established constitutional principles, defined operational architecture, and documented human interaction doctrine. Volume IV answers the engineering question: where do we begin, what do we build first, and how do we know when it works?

The objective is not to build the entire SHIELD ecosystem immediately. The full ecosystem – LumaShield, the marketplace, the Professional Shield network, the enterprise platform, the mobile applications, and the full AI operating system – represents years of iterative development. Attempting to build all of it in a first program would produce neither working software nor valid evidence of the architecture's correctness.

The objective is to build the smallest working system that proves the architecture. Specifically, the first engineering program targets the Translation Engine MVP and the Composition Engine MVP: the two foundational pipeline stages that together demonstrate that operational knowledge can be ingested from real enterprise artifacts, translated into structured semantic models, composed into deployable runtime specifications, and rendered as prototype adaptive experiences.

Core Principle: Build the smallest working system that proves the architecture. Every build decision should be evaluated against this constraint. If a component does not contribute to validating the core pipeline, it is out of scope for the MVP.

Volume IV defines the MVP boundary with precision, specifies each component workstream with acceptance criteria, establishes the schema-first engineering discipline, and provides a 90-day execution roadmap that produces the first evidence-backed Translation Package and Shield Runtime Specification from a real operational artifact.

Engineering Thesis

SHIELD engineering begins not with a broad platform build, but with one provable end-to-end workflow. The engineering thesis is precise: operational knowledge embedded in enterprise artifacts can be systematically extracted, modeled, composed, and rendered as an adaptive operational experience. The MVP exists to prove this thesis with real data, real connectors, and a real working prototype.

The thesis is validated when the following pipeline produces a working output from a real source artifact:



Each stage in this pipeline has a defined input, a defined output, and a schema contract that governs the transformation. The pipeline is the MVP. Proving the pipeline with a real enterprise artifact – from ingestion to prototype – is the first engineering milestone that validates the entire SHIELD architecture thesis.

1**Ingest**

Real enterprise artifact enters the pipeline with full provenance preserved

2**Translate**

Operational knowledge is extracted and modeled into structured semantic representations

3**Compose**

Context-aware runtime specification assembled from identity, intent, and operational context

4**Render**

Prototype adaptive experience generated from the Shield Runtime Specification

MVP Definition

Defining what the MVP is requires equal precision in defining what it is not. Scope discipline is an engineering requirement, not a strategic preference. The MVP boundary must be maintained to produce evidence quickly and avoid architectural drift before the core pipeline is validated.

The MVP Is Not

- Full LumaShield consumer application
- Full marketplace or distribution layer
- Full Professional Shield network
- Full enterprise platform
- Full mobile application
- Full AI operating system

The MVP Is

A working pipeline that can ingest a real operational artifact and generate a structured Shield-ready output – from source to prototype – with full provenance, validation, and engineering documentation preserved.

MVP Input Examples

Appenate Form JSON

Structured form definitions from enterprise mobile workflow platforms

SmartTag Workbook

Legacy workflow system definitions containing embedded operational intelligence

PDF SOP / Word Procedure

Unstructured operational documents with procedural and compliance content

Website / API Definition

Web-accessible operational resources and API contracts describing system behavior

MVP Output Package

01

Knowledge Corpus

Normalized source objects with full provenance

02

Operational Knowledge Extraction Report

Entities, actors, roles, events, rules, evidence, connectors

03

Operational Knowledge Model + Graph

Structured semantic model and relationship graph

04

Operational Context Package

Deployable context package with role-specific action sets

05

Shield Runtime Specification

Runtime-ready specification for prototype rendering

06

Prototype Prompt + Engineering Package

Canvas, Figma, and code scaffold prompts for prototype generation

MVP Case Study #001 — SmartTag Irving Oil

The first engineering case study is SmartTag Irving Oil. SmartTag was selected as the first MVP case because it proves a foundational claim of the SHIELD architecture: that legacy workflow systems contain hidden operational intelligence. The SmartTag system — built on Appenate and NoMorePaper — encodes employees, equipment, inspections, transactions, REST connectors, visibility rules, required-field logic, evidence requirements, and role-based behaviors in a structured workflow definition that the Translation Engine can systematically parse and model.

The SmartTag source artifact is an Appenate form JSON definition. It is a real enterprise artifact from a real industrial operation. Using it as the first test case means the MVP validation is grounded in operational reality, not a synthetic example. If the pipeline can ingest SmartTag and produce a validated Shield Runtime Specification, the architecture is proven on real evidence.

Structural Layer

Form controls, screens, sections, field types, and navigation structure embedded in the Appenate definition

Semantic Layer

Entities (Employee, Scaffold, Tag, Transaction), roles, events, and operational meaning encoded in labels and identifiers

Behavioral Layer

Visibility rules, required-field conditions, state transitions, and decision logic governing form behavior

Connector Layer

REST API endpoints, authentication patterns, request/response schemas, and credential references embedded in the workflow

- ❏ MVP Goal: Convert SmartTag into a complete Translation Package and Shield Runtime Specification. Every output from this case becomes a regression baseline for all future cases.

Engineering Principles

The SHIELD engineering program operates under twelve principles that govern every build decision, every schema definition, every connector specification, and every published operational asset. These principles are not aspirational — they are enforced through acceptance criteria, schema validation, security scanning, and human review gates.

1 Evidence before expansion

No component is expanded before its core function is validated against a real artifact with documented acceptance criteria.

2 Build from real artifacts

Every engineering target is grounded in a real operational source. Synthetic examples are only used for unit testing, not for architectural validation.

3 Preserve provenance

Every generated object traces back to its source artifact. Provenance is not optional metadata — it is a schema requirement for every output type.

4 Separate Translation from Composition

The Translation Engine and Composition Engine are independent systems with defined interface contracts. They are not co-mingled at the implementation level.

5 Keep the platform model-agnostic

Foundation models are interchangeable reasoning engines. No SHIELD schema, operational model, or runtime specification is coupled to a specific AI provider.

6 Treat connectors as governed assets

Every connector is versioned, permission-scoped, and subject to governance review before use in an operational context package.

1 Never embed secrets in operational definitions

Raw credentials, API keys, bearer tokens, and passwords must be detected and replaced with governed secret references before any package is published.

2 Validate before publishing

Schema validation, semantic validation, security scanning, and human review must all pass before any operational asset changes publication state.

3 Keep schemas versioned

Every schema carries a version identifier. Breaking schema changes require a new major version. Backward-compatible changes increment the minor version.

4 Every case study becomes regression evidence

Outputs from each case study are committed to the regression suite. Future changes to the pipeline must not break existing validated outputs.

5 Human review required for high-consequence workflows

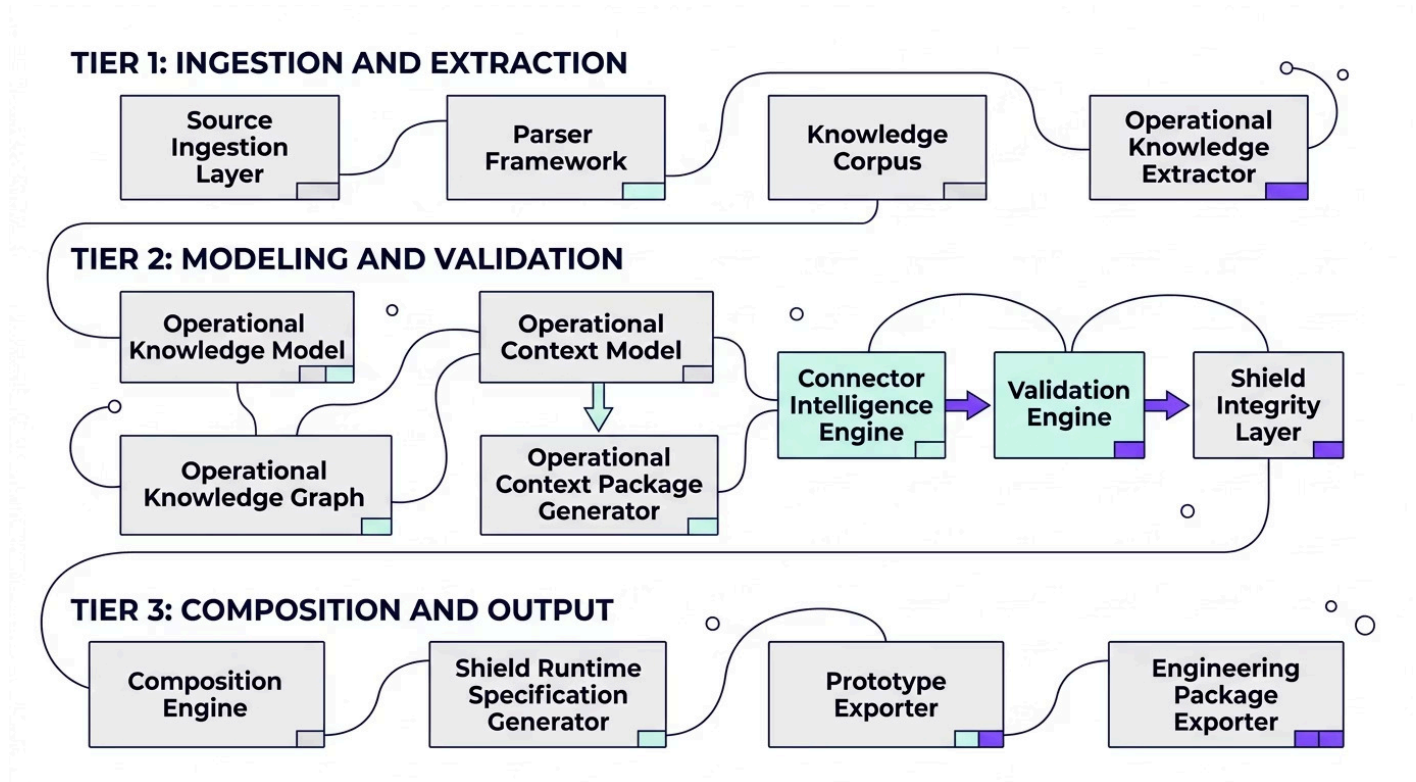
Automated extraction and composition are tools that assist human judgment. High-consequence operational packages require explicit human reviewer approval before publication.

6 The architecture must remain portable

No component may create a lock-in dependency on a specific cloud provider, model vendor, or proprietary runtime. Portability is an architectural constraint, not a preference.

Core System Components

The SHIELD engineering program is composed of fifteen major components spanning the Translation Engine and Composition Engine layers. Each component has a defined purpose, a defined interface contract, and an independent workstream. Together they form the complete pipeline from source artifact to adaptive Shield experience.



The three tiers reflect the separation of concerns that is central to the SHIELD engineering discipline. Tier 1 handles ingestion and extraction – converting raw enterprise artifacts into structured semantic objects. Tier 2 handles modeling and validation – building the Operational Knowledge Model, the graph, and the context package while enforcing security and governance constraints. Tier 3 handles composition and output – assembling runtime specifications and generating prototype artifacts from the validated operational intelligence produced by Tiers 1 and 2.

Repository Structure

The SHIELD engineering program lives in a single GitHub monorepo: `shield-generator/`. The repository is organized by functional domain, not by team or feature. This organization enforces the separation of Translation Engine and Composition Engine concerns at the filesystem level and makes the architecture legible to every contributor from day one.

Translation Engine

```
translation-engine/
  ingestion/
  extraction/
  ontology/
  graph/
  context/
  validation/
  exporters/
```

Composition Engine

```
composition engine/
  identity/
  intent/
  context-resolution/
  capability-selection/
  runtime-spec/
  rendering/
```

Connectors

```
connectors/
  appenate/
  office/
  pdf/
  web/
  api/
  quickbooks/
  sharepoint/
```

Schemas

```
schemas/
  source/
  corpus/
  okm/
  okg/
  ocm/
  ocp/
  runtime/
  assets/
```

Cases

```
cases/
  001-smarttag/
  source/
  extracted/
  graph/
  context/
  runtime/
  prototypes/
  reports/
```

Documentation

```
docs/
  canon/
  architecture/
  engineering/
  research/
  cases/
```

Tools and Tests

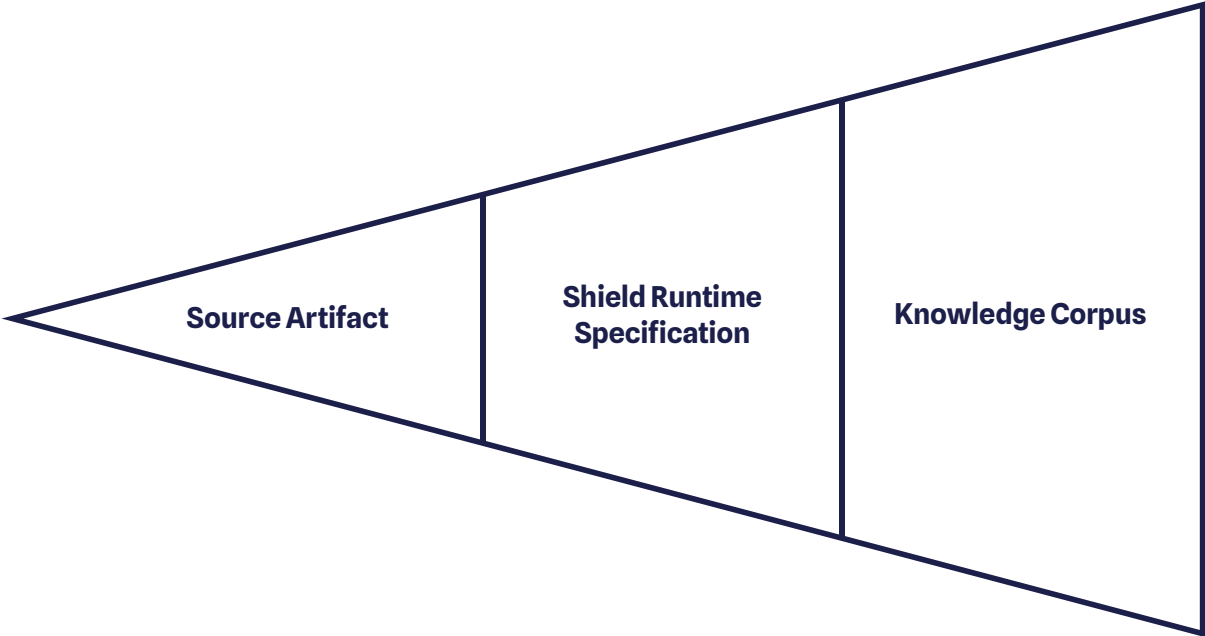
```
tools/
  cli/
  notebooks/
  scripts/

tests/
  unit/
  integration/
  regression/
```

- ❑ The `cases/` directory is the regression suite. Every case study committed here becomes a permanent validation baseline. No pipeline change is considered safe until it passes all existing case regression tests.

Data Flow Architecture

The SHIELD data pipeline is a strictly ordered sequence of transformations. Each stage receives a defined input schema, applies a defined transformation, and produces a defined output schema. No stage consumes outputs from a stage it does not directly succeed. This ordered dependency model is enforced by the schema contracts and the validation engine at each transition point.

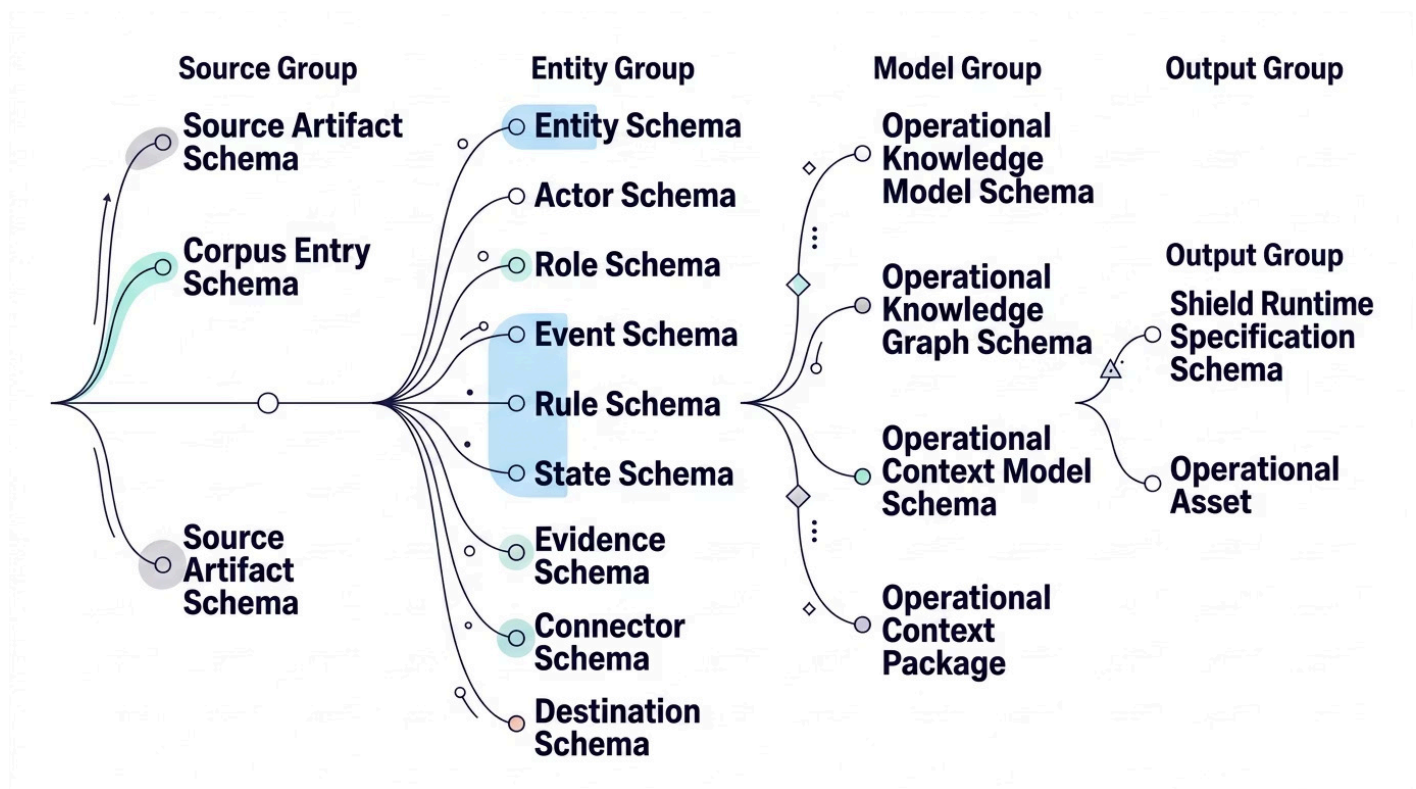


01	02	03
Source Artifact → Parser The parser reads the raw enterprise artifact (JSON, Excel, PDF, Word, API response) and produces a Normalized Source Object that conforms to the Source Artifact Schema.	Parser → Knowledge Corpus Normalized Source Objects are indexed into the Knowledge Corpus with full provenance metadata, source type classification, and security scan findings attached.	Corpus → Operational Knowledge Model The Extractor reads corpus entries and identifies entities, actors, roles, events, states, rules, evidence requirements, connectors, and permissions, producing a structured OKM.
04	05	06
OKM → Operational Knowledge Graph The Graph Generator converts the OKM into a semantic relationship graph with typed nodes, typed edges, state transitions, and source references preserved on every element.	OKG → Operational Context Package The Context Generator produces a deployable OCP from the OKG, incorporating role-specific action sets, evidence requirements, connector recipes, and validation metadata.	OCP → Shield Runtime Specification The Composition Engine converts the OCP into a runtime-ready Shield specification defining Dial configuration, Pad definitions, Pearl behavior, Ring states, and voice prompts.
07		
Runtime Spec → Prototype / Renderer The Prototype Exporter generates Canvas prompts, Figma prompts, code scaffold prompts, and HTML prototype artifacts from the validated Shield Runtime Specification.		

Schema Strategy

SHIELD engineering is schema-first. Every component in the pipeline is defined by the schema contracts it consumes and produces. Schemas are the authoritative contract between architecture and implementation – they define what a valid output looks like before a single line of transformation logic is written. This approach prevents implementation drift, enables independent component development, and makes validation deterministic.

All schemas are versioned, stored in the `schemas/` directory of the monorepo, and referenced by version identifier in every generated artifact. Breaking schema changes require a new major version. Backward-compatible additions increment the minor version. The schema version is part of the provenance metadata on every generated object.



Engineering Law: Schemas are the contract between architecture and implementation. No component may consume or produce data that does not conform to its governing schema. Schema violations are build failures, not warnings.

Corpus Ingestion Engine

The Corpus Ingestion Engine is the entry point of the Translation Engine pipeline. Its purpose is to read enterprise artifacts in any supported format, normalize them into structured source objects, index them into the Knowledge Corpus, and attach complete provenance and security metadata to every corpus entry.

Supported Inputs

- JSON (Appenate form definitions, API responses)
- Excel / CSV (workflow workbooks, data exports)
- PDF (SOPs, compliance documents)
- Word (procedures, policy documents)
- Website (web-accessible operational resources)
- API response (structured system outputs)

Outputs

- Source inventory manifest
- Normalized source objects (per Source Artifact Schema)
- Corpus entries (per Corpus Entry Schema)
- Provenance map linking every object to its source
- Security findings report (credentials, sensitive data)

Acceptance Criteria — Case #001 SmartTag

Ingest SmartTag JSON

Successfully parse the complete Appenate form definition without data loss

Preserve Provenance

Every corpus entry links back to the exact source file, version, and ingestion timestamp

Detect Credentials

Identify embedded passwords, API keys, tokens, and hard-coded URLs in source artifacts

Export Corpus JSON

Produce a valid corpus export conforming to the Corpus Entry Schema with version identifiers

Generate Ingestion Report

Produce a human-readable ingestion summary with statistics, findings, and quality flags

Operational Knowledge Extractor

The Operational Knowledge Extractor is the semantic core of the Translation Engine. It reads normalized corpus entries and extracts the operational meaning encoded in the source artifact – converting structural and textual content into typed operational objects that conform to the entity schemas defined in the schema strategy.

Extraction is performed using a combination of rule-based pattern matching, AI-assisted semantic interpretation, and schema-governed validation. Every extracted object carries a confidence score and a source reference. Low-confidence extractions are flagged for human review before they can be promoted to the Operational Knowledge Model.

Entities & Actors

Named operational objects (Employee, Scaffold, Tag, Inspection) and the roles that interact with them

Events & States

Operational events (scan, inspect, submit, approve) and the state transitions they trigger

Rules & Constraints

Visibility rules, required-field conditions, permission checks, and decision logic embedded in the workflow

Connectors & Evidence

REST endpoints, authentication patterns, evidence requirements, and external system relationships

Acceptance Criteria — Case #001 SmartTag

- Identify all SmartTag operational events (scan, validate, inspect, submit)
- Extract all REST connector definitions with endpoint schemas
- Extract all visibility and required-field rules with source references
- Extract evidence requirements for each operational event
- Map all form controls to typed operational objects per entity schemas
- Generate an extraction report with confidence scores and human review flags

Operational Knowledge Graph Generator

The Operational Knowledge Graph Generator converts the structured Operational Knowledge Model into a semantic relationship graph. The graph makes explicit the relationships between entities, events, actors, evidence requirements, connectors, and state transitions that are implicit in the flat OKM representation. The graph is the analytical substrate that enables context-aware composition in the Composition Engine layer.

Graph Inputs

- Operational Knowledge Model (OKM)
- Entity definitions and relationships
- Event sequences and state transitions
- Connector and evidence associations

Graph Outputs

- Typed graph nodes (entity, event, actor, evidence, connector)
- Typed graph edges (triggers, requires, produces, connects)
- State transition map
- Graph JSON export
- Visual graph diagram

Acceptance Criteria — Case #001 SmartTag

Generate SmartTag Graph

Produce a complete OKG from the SmartTag OKM with all node and edge types populated

Represent Core Entities

Employee, Scaffold, Inspection, Evidence, and Transaction nodes must be present with correct relationship edges

Export Graph JSON

Produce a valid graph export conforming to the Operational Knowledge Graph Schema

Generate Visual Diagram

Produce a human-readable graph visualization for review and documentation

Preserve Source References

Every node and edge must carry a provenance reference to the source corpus entry

Operational Context Package Generator

The Operational Context Package Generator produces the deployable Operational Context Package – the primary interface between the Translation Engine and the Composition Engine. The OCP is a self-contained, validated, role-annotated operational intelligence package that defines what actions are available, what evidence is required, what connectors are accessible, and what permissions govern each combination of identity, role, and operational state.

The OCP is the unit of operational deployment in the SHIELD architecture. It is versioned, governed, and subject to the full validation and publication workflow before it can be consumed by the Composition Engine. An OCP that has not passed validation and human review cannot be promoted to sandbox or published state.

Inputs

- Operational Knowledge Model
- Operational Knowledge Graph
- Rules, states, evidence definitions
- Connector definitions
- Permission scopes

Outputs

- OCP JSON (versioned)
- Context summary
- Available and blocked action sets
- Evidence requirements by role
- Connector recipes
- Validation metadata

Acceptance Criteria— Case #001 SmartTag

- Generate SmartTag Transaction OCP conforming to Operational Context Package Schema
- Include role-specific action sets for all identified roles
- Include evidence requirements for each operational event
- Include connector recipes for all identified REST connectors
- Include confidence metadata on all extracted and composed elements

Shield Runtime Specification Generator

The Shield Runtime Specification Generator is the final stage of the Translation Engine and the primary input to the Composition Engine rendering layer. It converts a validated Operational Context Package into a runtime-ready Shield specification that defines every aspect of the adaptive interface: the Dial configuration, the Pad definitions, the Pearl behavior, the Ring state logic, the Context Window layout, voice prompts, and the complete runtime state map.

The Shield Runtime Specification is not a UI design document. It is a machine-readable, schema-governed operational specification that any compliant Shield renderer can consume and produce a working adaptive experience from. The specification is device-mode-aware and role-specific – a different runtime specification is produced for different identity/role combinations operating in different device modes.

Inputs

- Operational Context Package
- Identity and role assumptions
- Target device mode

Outputs

- Dial configuration
- Pad definitions (primary actions)
- Pearl behavior (contextual triggers)
- Ring state definitions
- Context Window layout model
- Voice prompts
- Blocked action definitions
- Runtime state map

Acceptance Criteria — SmartTag

- Generate complete SmartTag Shield Runtime Spec from validated OCP
- Include Scan, Inspect, Measure, and Materials Pad definitions
- Define Pearl behavior for each operational state
- Define Ring state rules for tag validation and inspection completion
- Export valid YAML or JSON conforming to Shield Runtime Specification Schema

Connector Intelligence Engine

The Connector Intelligence Engine identifies, normalizes, and governs external system relationships embedded in enterprise source artifacts. Connectors — REST APIs, webhooks, email integrations, document outputs, database connections, SaaS APIs, authentication services, file storage systems, and reporting endpoints — are first-class assets in the SHIELD architecture. They cannot be ad-hoc implementation details; they must be governed, versioned, permission-scoped, and secret-safe before they appear in any Operational Context Package.



REST API & Webhook

Structured HTTP endpoints and event-driven integrations with defined request/response schemas and authentication requirements



Database & SaaS API

Direct data source connections and cloud platform integrations requiring permission scoping and credential governance



Email & Document Output

Notification delivery, report generation, and document distribution connectors with audit and retry requirements



Authentication & Storage

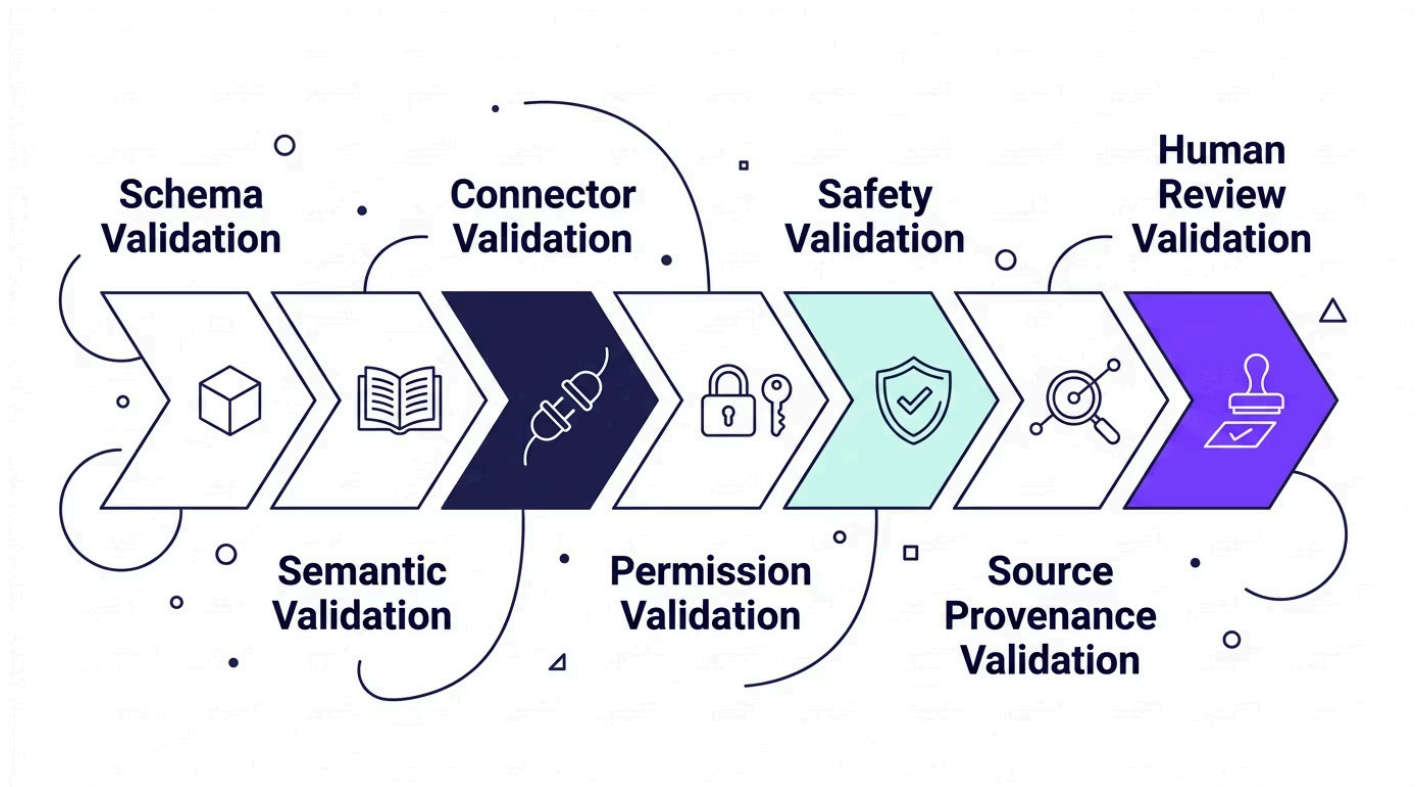
Identity provider integrations and file storage connectors with environment-aware credential management

Acceptance Criteria — Case #001 SmartTag

- Extract SmartTag REST login endpoint with authentication schema
- Extract employee and tag validation endpoints with request/response schemas
- Extract transaction submission endpoint with payload schema
- Detect all embedded credentials and flag as security findings
- Convert raw credentials into `governed_secret` reference objects
- Generate a Connector Intelligence Report for human review

Validation Engine

The Validation Engine is the quality gate for every output produced by the Translation Engine pipeline. Nothing transitions from one publication state to the next without passing the validation checks defined for its schema and content type. Validation is not a final step – it is embedded at every stage transition and enforced by the pipeline orchestration layer.



Acceptance Criteria — Case #001 SmartTag

- Validate SmartTag OCP against Operational Context Package Schema – zero schema violations allowed
- Flag all low-confidence mappings for human review before publication
- Flag all security findings identified by ENG-008 before package promotion
- Require explicit human reviewer approval before any package transitions to approved or published state

❏ The Validation Engine does not approve packages – it gates them. Human review is the final gate. No automated system may approve a high-consequence operational package for publication without a human reviewer signature on record.

Security and Secret Detection

The Security and Secret Detection component operates as a mandatory scan at the ingestion stage, at the extraction stage, and at the OCP generation stage. It prevents unsafe operational packages from reaching composition, publication, or prototype rendering by detecting all forms of embedded credentials, sensitive personal data, and unsafe connector patterns before they can propagate through the pipeline.

Detection Targets

- Passwords and passphrases
- API keys and bearer tokens
- Usernames in endpoint definitions
- Hard-coded URLs containing credentials
- Exposed authentication credentials
- Unsafe connector patterns
- Sensitive personal data (PII)
- Regulated data categories

Remediation Actions

- Create a structured security finding object
- Block pipeline progression for the affected object
- Generate a human-readable remediation recommendation
- Replace raw credential with a `governed_secret` reference object
- Log finding in the provenance and audit record

Acceptance Criteria — Case #001 SmartTag

- Detect all embedded SmartTag credentials (login endpoint credentials, API tokens)
- Create structured security finding objects for each detected credential
- Generate a remediation recommendation document for human reviewer
- Replace all detected raw credentials with `governed_secret` reference objects in the corpus and OCP

Provenance and Versioning Layer

The Provenance and Versioning Layer is not a discrete component in the pipeline – it is a cross-cutting constraint that applies to every object generated in the system. Every corpus entry, every extracted entity, every OKM object, every OKG node and edge, every OCP, every Shield Runtime Specification, and every prototype artifact must carry complete provenance metadata. The ability to trace any generated object back to its original source is a non-negotiable architectural requirement.

Source Identity

source_id – unique identifier for the source artifact. **source_version** – version of the source at ingestion time. **ingestion_timestamp** – ISO 8601 timestamp of ingestion.

Extraction Metadata

extraction_timestamp – when the object was extracted. **model_tool_used** – the AI model or tool version that performed extraction. **confidence_score** – numeric confidence in the extraction result.

Governance Metadata

reviewer – human reviewer identity for approved objects. **validation_state** – current validation status. **change_history** – ordered list of all modifications. **rollback_path** – reference to previous valid version.

Acceptance Criteria — Case #001 SmartTag

- Every SmartTag output object links back to the original source artifact by `source_id` and `source_version`
- Every generated object carries the full required provenance metadata set
- Every schema carries a version identifier and a `schema_version` field in generated outputs
- Every package can be fully regenerated from the original source artifact using the recorded provenance chain

Identity Resolution Engine

The Identity Resolution Engine is the first component of the Composition Engine layer. It resolves the active identity context for a composition session – determining who the user is, what roles and permissions they hold, what context they have adopted, and what trust state governs their interaction with the system. Identity resolution precedes all other composition steps: capability selection, context resolution, and runtime specification generation are all downstream of identity.

SHIELD supports multiple identity modes, reflecting the diversity of operational contexts in which the system is deployed. The identity layer must support every mode without requiring a different composition path for each – identity mode is an input parameter, not an architectural branch.

Anonymous

No persistent identity. Limited to publicly available operational context. No personal or professional data visible.

Personal (Luma ID)

Personal identity context with access to personal operational packages, home context, and personal Shield types.

Professional (Shield ID)

Professional identity context with role-based permissions, employer-governed access, and professional Shield types.

Organization

Organization-scoped identity for multi-user operational contexts with organization-level permission governance.

Temporary / Adopted

Time-limited identity context for delegated access, temporary role adoption, and supervised operational contexts.

Outputs of the Identity Resolution Engine include the active identity layer, visible and hidden identity attributes, trust state classification, and the permission scope that governs all downstream composition decisions.

Intent Resolution Engine

The Intent Resolution Engine interprets what the user is attempting to accomplish given their current identity, context, and input signal. Intent resolution is multimodal — it must correctly interpret voice commands, tap interactions, contextual triggers from the environment, workflow events, beacon signals, and nudge activations. The output is a structured intent object that carries the resolved intent type, a confidence score, any required clarification prompts, and the predicted next action.

Input Signals

- Voice command
- Tap interaction
- Current operational state
- Active workflow event
- Beacon signal
- Nudge activation

Output: Intent Object

- Resolved intent type
- Confidence score
- Required clarification prompts (if confidence below threshold)
- Likely next action prediction

Acceptance Criteria

SmartTag: Validate Scaffold

Correctly resolve intent as scaffold validation event, select scan capability, load tag validation connector

HomeShield: Furnace Won't Start

Correctly resolve intent as home system diagnostic, select diagnose capability, load relevant service context

Professional Shield: Ask Lawyer Question

Correctly resolve intent as professional consultation request, select expert connection capability, load professional Shield context

Context Resolution Engine

The Context Resolution Engine determines what matters now – given the resolved identity, the resolved intent, the available Operational Context Package, and all available contextual signals (location, device state, active events, memory, and device mode). It produces the current operational context that governs capability selection and runtime specification generation.

Context resolution is the most computationally complex component of the Composition Engine. It must integrate signals from multiple sources with different reliability levels and different update frequencies, and produce a coherent, ranked context model that correctly prioritizes the most operationally relevant information for the current moment. The output drives every subsequent composition decision.

Identity + Role
Who is acting and what permissions govern their actions

OCP
The governing Operational Context Package for the current operational domain



Intent

What the user is trying to accomplish right now

Location + State

Physical and operational state signals from the environment

Memory + Events

Active events, recent history, and relevant operational memory

Acceptance Criteria

- Produce operational context for SmartTag scaffold inspection event with correct role scope
- Produce operational context for HomeShield service request with home system context
- Produce operational context for Professional Shield client interaction with professional identity scope

Capability Selection Engine

The Capability Selection Engine selects the smallest useful set of active capabilities for the current composition session. This is the component that enforces the discipline of minimum viable interface – rather than presenting every possible action to the user, it identifies the four primary Pad actions, the Pearl action, and the blocked actions that are most operationally relevant given the current identity, intent, context, and role. Cognitive load reduction is a design constraint, not a preference.

Scan

QR, NFC, barcode, tag, and document scanning capabilities

Inspect

Structured inspection workflows with evidence capture and compliance checking

Diagnose

System and equipment diagnostic workflows with guided troubleshooting

Ask Expert

Connection to human expertise with context-packaged handoff

Beacon / Nudge

Attention signals and proactive operational prompts

Submit / Approve

Transaction completion, approval workflows, and compliance sign-off

Pay / Schedule

Payment initiation and scheduling capabilities for service contexts

Capture Evidence

Photo, video, measurement, and document evidence capture workflows

Acceptance Criteria

- Select exactly four primary Pad actions for any given composition session
- Define the Pearl action – the single highest-priority contextual trigger
- Define blocked actions with clear blocking reasons for the current role and state
- Support role-specific composition – different roles produce different capability sets from the same OCP

Shield Runtime Generator

The Shield Runtime Generator is the final Composition Engine component. It assembles all resolved composition inputs – identity layer, intent object, operational context, selected capabilities – into a complete, schema-validated Shield Runtime Specification that a compliant renderer can consume to produce an interactive adaptive experience. The runtime specification is device-mode-aware, role-specific, and fully self-contained.

Runtime Specification Components

- Runtime JSON or YAML (versioned, schema-validated)
- Dial configuration (navigation and orientation model)
- Pad configuration (four primary action definitions)
- Pearl configuration (contextual trigger and priority action)
- Ring state definitions (operational state machine)
- Context Window model (information display layout)
- Device layout model (form factor adaptation)
- Voice prompts (all voice-addressable actions)

Acceptance Criteria

- Generate a valid runtime specification conforming to Shield Runtime Specification Schema
- Specification is directly renderable by the prototype tools (Gemini Canvas, Figma, HTML)
- Specification is portable across all supported device modes
- Role-specific variants are generated from a single OCP without manual customization

Prototype Exporter

The Prototype Exporter translates validated Shield Runtime Specifications into the prompt and scaffold formats consumed by prototype generation tools. It is the bridge between the schema-governed engineering pipeline and the interactive prototype artifacts that demonstrate the architecture to engineers, researchers, investors, and stakeholders. The Prototype Exporter does not generate the prototypes itself – it generates the precisely structured inputs that prototype tools need to produce high-fidelity interactive demonstrations.



Gemini Canvas Prompt

Structured natural language prompt optimized for Gemini Canvas interactive prototype generation, including full runtime spec context



Figma Prompt

Component-level design prompt for Figma prototype construction, referencing Shield design system elements



Cursor / Claude Code Prompt

Engineering-level code scaffold prompt for Cursor or Claude Code to generate an HTML or React prototype from the runtime spec



Gamma Summary Prompt

Documentation prompt for generating an executive-ready Gamma presentation from the engineering package

Acceptance Criteria

- Generate complete SmartTag prototype package (all prompt types + HTML scaffold + JSON runtime spec)
- Generate complete Professional Shield prototype package
- Generate complete HomeShield prototype package

GEMS MVP — The Shield Generator

GEMS (the Shield Generator) is the orchestration layer for the complete Translation and Composition pipeline. GEMS v0.1 is the first implementation of an end-to-end orchestration interface that accepts organizational inputs and drives the pipeline from source ingestion through to a complete engineering package ready for prototype generation, research documentation, and executive presentation.

GEMS begins with one initial user question: **What organization are we translating?** From this starting point, GEMS collects the inputs required to initialize the pipeline and orchestrates every subsequent step, from source ingestion through corpus generation, knowledge extraction, graph generation, context packaging, runtime specification, and prototype export.

GEMS Inputs

- Organization name
- Website URL
- Source documents and artifacts
- API definitions
- Operational goals

GEMS Outputs

- Organizational Context Package
- Technology Stack assessment
- Connector Graph
- Operational Assessment
- Knowledge Graph
- Shield Specification
- Gemini Canvas Prompt
- Figma Prompt
- Cursor Prompt
- GitHub Scaffold
- Executive Presentation

GEMS v0.1 does not need to be a polished product interface. It needs to be a functional orchestration layer — a CLI tool, a Python notebook, or a simple web interface — that reliably drives the full pipeline for a given organization and produces all required outputs in structured, versioned, schema-valid form. GEMS is the proof-of-concept implementation vehicle for the entire Translation and Composition Engine architecture.

Development Environment

The SHIELD engineering development environment is assembled from tools selected for reliability, ecosystem maturity, and alignment with the schema-first, model-agnostic engineering principles of the program. Tool selections are recommendations, not mandates – teams may substitute equivalents that conform to the same interface contracts and schema standards.

Source Control

GitHub – monorepo hosting, GitHub Actions for CI, GitHub Issues for workstream tracking

Engineering IDEs

VS Code or Cursor for primary development. Cursor AI-assisted engineering for scaffold generation and code review.

Backend Services

Python for ingestion, extraction, and schema validation. FastAPI or equivalent for backend service endpoints. Pydantic for schema enforcement.

Runtime & Interface

TypeScript for runtime and interface layers. Shared JSON schemas between Python and TypeScript implementations.

Data Storage

PostgreSQL for relational data. Neo4j or equivalent graph database for OKG experiments. Object storage for source artifacts.

Prototype & Documentation

Gemini Canvas for interactive prototypes. Figma for design system. Gamma for executive documentation. NotebookLM for curated knowledge.

Technical Stack Options

The SHIELD engineering program does not mandate a single technology stack. Two primary stack configurations are appropriate for different layers of the system, and the recommended approach combines both – using each where it is most suited. The shared JSON schema contracts between layers ensure that stack choice at one layer does not constrain stack choice at another.

Option A — Python-First MVP

Best for:

- Source ingestion pipelines
- AI-assisted extraction workflows
- Schema validation with Pydantic
- Research notebooks and experiments
- Graph generation with networkx or Neo4j drivers
- CLI tooling and script automation

Python's AI and data ecosystem makes it the natural choice for the Translation Engine workstreams.

Option B — TypeScript-First Runtime

Best for:

- Web runtime and interactive prototypes
- Shield renderer implementations
- Frontend composition interface
- Shared schema validation in browser context
- Type-safe schema consumption

TypeScript's type system and web ecosystem make it the natural choice for the Composition Engine runtime and prototype renderer layers.

Recommendation: Python for the Translation Engine (ingestion, extraction, ontology, graph, validation). TypeScript for the Composition Engine runtime and prototype renderer. Shared JSON schemas between both – the schema is the contract, not the language.

Model Gateway

The SHIELD Model Gateway defines how the engineering program accesses foundation model reasoning capabilities without creating coupling between the operational architecture and any specific AI provider. Foundation models are interchangeable reasoning engines in the SHIELD architecture – they perform extraction, semantic interpretation, and composition tasks, but they do not own the schemas, operational models, context packages, runtime specifications, governance rules, or interaction grammar that define the system. SHIELD owns those assets permanently, regardless of which model providers are used at any given time.



OpenAI

GPT-family models for extraction, semantic interpretation, and structured output generation tasks



Anthropic

Claude models for long-context document analysis, extraction, and code generation tasks



Gemini

Gemini models for multimodal analysis, canvas prototype generation, and research workflows



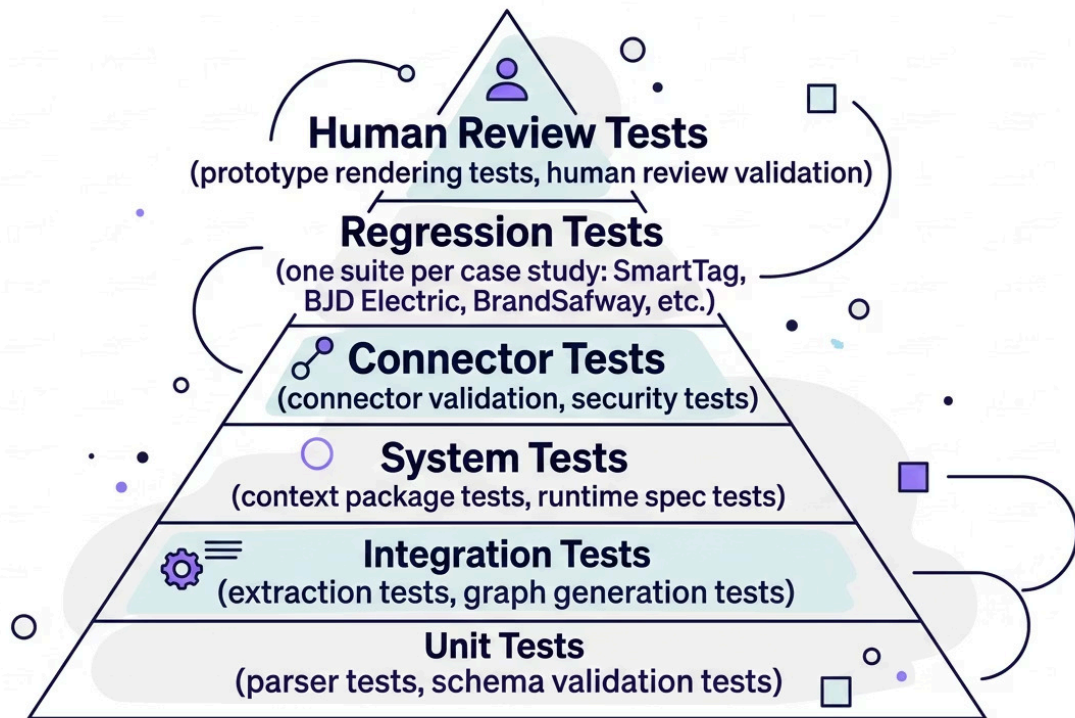
Azure OpenAI / Local

Enterprise-sovereign deployment options and locally-hosted models for regulated operational contexts

The Model Gateway implementation must abstract all model-specific API calls behind a common interface contract. Swapping providers must require no changes to Translation Engine or Composition Engine logic – only configuration changes to the gateway layer. Model selection for each task type is a configuration parameter, not a hardcoded implementation decision.

Testing Strategy

The SHIELD testing strategy is built on the same evidence-before-expansion principle that governs the engineering program. Tests are organized by pipeline stage, and every case study produces a new layer of regression evidence that must be preserved across all future pipeline changes. A pipeline change that breaks an existing case study regression is a breaking change, regardless of whether it improves other metrics.



Parser Tests

Validate that every supported source format is parsed correctly into a normalized source object

Schema Validation Tests

Assert that every pipeline output conforms to its governing schema – zero tolerance for schema violations

Extraction Tests

Validate entity, actor, event, rule, and connector extraction against known-good outputs from committed case studies

Security Tests

Assert that the security scanner detects all known credential patterns and produces correct remediation outputs

Regression Tests

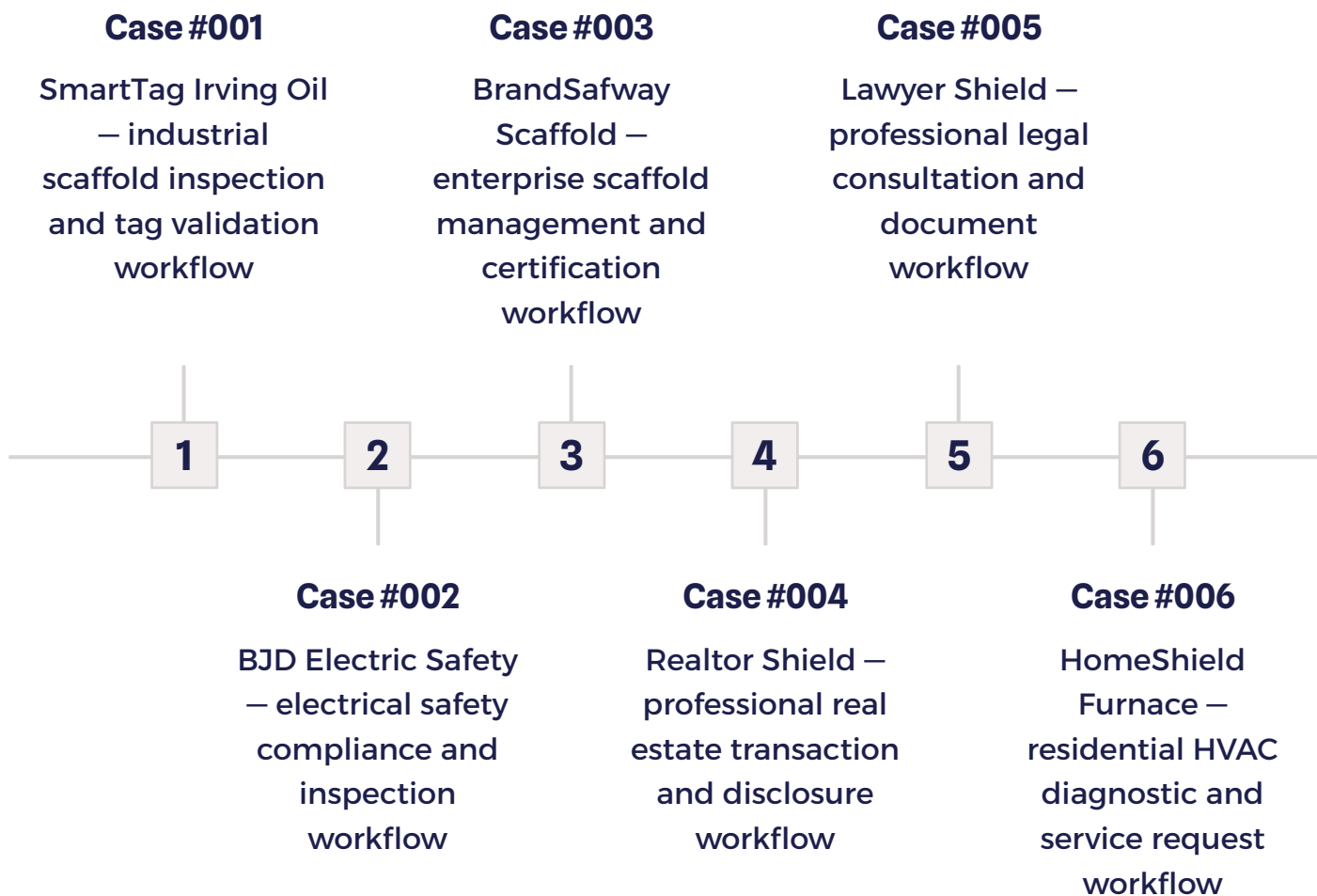
Confirm that every committed case study output is reproduced exactly by the current pipeline version

Prototype Rendering Tests

Validate that runtime specifications produce correct prompt structures for all supported prototype generation tools

Case Study Regression Suite

The case study regression suite is the engineering program's primary quality enforcement mechanism. Every case study that completes the full pipeline – from source ingestion through prototype export – is committed to the `cases/` directory of the monorepo with all expected outputs preserved. Future pipeline changes must reproduce these expected outputs exactly. The suite grows with each new case study processed by GEMS.



Case Study Structure

Every case study in the regression suite must include: source artifacts (original files committed verbatim), expected extraction report, expected OKM, expected OKG (graph JSON and visual), expected OCP, expected Shield Runtime Specification, expected prototype output package, and validation notes documenting any human review decisions made during processing.

API and Connector Strategy

Connectors in the SHIELD architecture are not ad-hoc integration code – they are governed operational assets with defined schemas, versioning, permission scopes, and publication states. The connector strategy ensures that every external system relationship identified in a source artifact is converted into a reusable, testable, permission-aware connector asset before it is included in any Operational Context Package.

Modular & Versioned

Each connector is an independent, versioned asset. Connector changes do not require changes to the OCP or runtime specification that references them – only the connector version reference is updated.

Permission-Aware

Every connector defines its required permission scope. The Validation Engine enforces that no OCP grants connector access beyond the permission scope granted by the governing identity and role context.

Environment-Aware

Connectors define environment-specific configurations (development, sandbox, production). No connector can expose production credentials in a non-production context.

Secret-Safe

Raw credentials never appear in a connector definition. All authentication values are referenced via `governed_secret` objects that resolve at runtime through the secret management layer.

Connector Package Schema

Field	Type	Description
<code>connector_id</code>	string	Unique versioned connector identifier
<code>auth_method</code>	enum	bearer, apikey, oauth2, basic, none
<code>endpoint_map</code>	object	Named endpoint definitions with path and method
<code>request_schema</code>	JSON Schema	Schema governing request payload structure
<code>response_schema</code>	JSON Schema	Schema governing expected response structure
<code>permission_scope</code>	array	Required permission grants for connector use
<code>secret_references</code>	array	References to governed secret objects – never raw values
<code>error_handling</code>	object	Error codes, retry rules, and fallback behaviors
<code>audit_required</code>	boolean	Whether connector calls require audit logging

Security Architecture

Security is a first-class architectural constraint in the SHIELD engineering program, not a post-implementation addition. The security architecture spans every layer of the pipeline – from source artifact ingestion through operational package publication and runtime specification delivery. Every component must be designed with the security requirements defined here as non-negotiable constraints, not optional features.

Engineering Law: No operational package should contain raw secrets. This constraint is enforced at the schema level, the validation level, and the publication gate. Any operational package that fails this constraint is blocked from promotion to any publication state above draft.

Identity & Permission Controls

Role-based access control enforced at every pipeline stage. Least privilege access for all service accounts. Tenant separation for multi-organization deployments.

Secret Management

All credentials managed through a secret management system. No raw secrets in schemas, operational definitions, connector packages, or OCP outputs. All references via `governed_secret` objects.

Source Artifact Classification

Every source artifact is classified by sensitivity level at ingestion. Classification governs storage permissions, access controls, and distribution restrictions on all downstream generated objects.

Audit Logging

All pipeline operations, publication state changes, human review decisions, and connector calls are logged with full provenance to a tamper-evident audit record.

Connector Isolation

Connectors execute in isolated contexts. A connector failure or security event in one connector cannot affect other connectors or the operational integrity of the composition pipeline.

Privacy-Preserving Memory

Operational memory in the Composition Engine layer must preserve privacy by design. Personal and professional identity data is never mixed without explicit permission grants.

Governance Architecture

Every generated operational asset in the SHIELD system exists within a defined governance lifecycle. Assets are not simply created and used – they are created, extracted, validated, reviewed, approved, published, and eventually deprecated or archived. The governance architecture defines the publication states, the transition rules, and the metadata requirements that apply to every operational asset at every stage of its lifecycle.



Required Asset Metadata

Every operational asset must carry: owner identifier, asset version, validation status, source provenance reference, permission scope definition, reviewer identity (for validated and above states), publication state, and rollback path reference. Assets without complete metadata cannot transition beyond draft state.

MVP Build Sequence

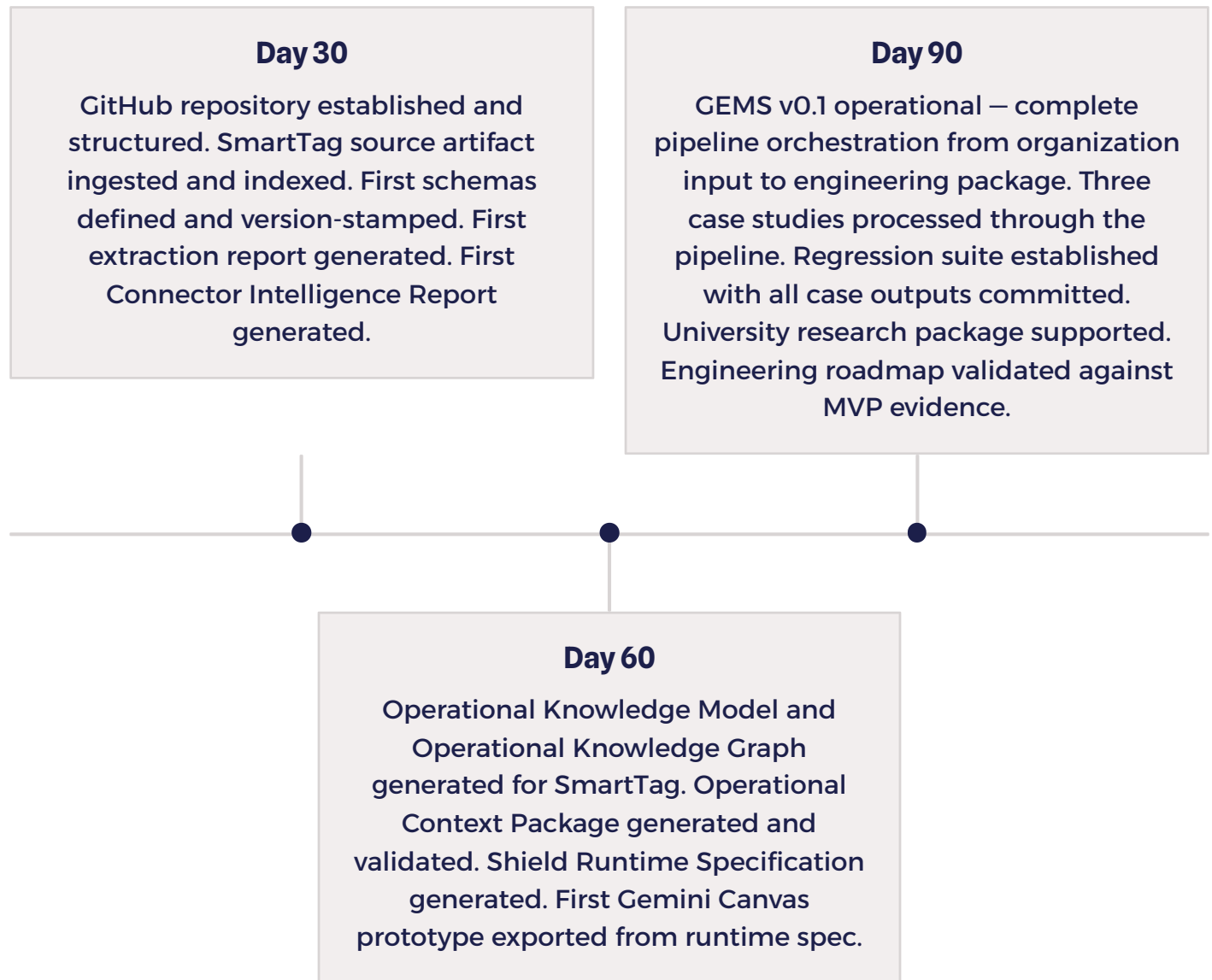
The MVP build sequence is organized into five ordered phases. Each phase has a defined set of deliverables and a clear dependency on the preceding phase. No phase begins until the preceding phase has produced its committed deliverables. This sequencing enforces the evidence-before-expansion principle at the program execution level.



Phase	Primary Deliverable	Acceptance Gate
Phase 1	SmartTag corpus in Knowledge Corpus	Schema-valid corpus JSON, ingestion report generated
Phase 2	SmartTag OKM and OKG	Schema-valid OKM + OKG, Connector Intelligence Report
Phase 3	SmartTag OCP + Validation Report	Schema-valid OCP, security report, human review flagged
Phase 4	Shield Runtime Spec + Prototype Package	Valid runtime spec, all prototype prompts generated
Phase 5	Regression suite + Engineering roadmap	All case #001 outputs committed to regression suite

30 / 60 / 90 Day Plan

The 90-day execution plan translates the build sequence into time-bound milestone commitments. Milestones are defined by engineering outputs, not by effort — the question is not whether the team worked hard, but whether the committed outputs exist and pass their acceptance criteria. Each milestone period closes with a documented review against acceptance criteria before the next period begins.



- The 90-day plan is evidence-driven: at Day 90, the engineering team must be able to demonstrate a live pipeline run from a new organizational input to a prototype output in a single session. The demonstration is the acceptance criterion for the 90-day milestone.

Team Roles

The SHIELD engineering program requires a set of roles that spans architecture, AI engineering, ontology design, full-stack development, interaction design, security, research, and documentation. Not all roles need to be filled by separate individuals in the initial program – roles can be combined, especially in a research or early-stage team context – but each responsibility must be owned by someone on the team.



Founder / Product Architect

Owens the SHIELD architecture canon, ensures all engineering decisions are consistent with Volumes I-III, sets MVP boundaries, and manages research alignment. Final decision authority on architecture questions.



Research Engineer

Drives notebook-based experiments for extraction, graph generation, and context modeling. Produces evidence from case studies that feeds into the regression suite and research documentation.



AI Engineer

Builds and maintains the Model Gateway, develops extraction prompts, implements model-agnostic abstraction layer, and manages AI tool integration across the Translation and Composition pipelines.



Ontology Engineer

Defines and maintains the entity schemas, the operational knowledge model, and the graph structure. Ensures semantic consistency across all extraction outputs and schema versions.



Full-Stack Developer

Implements Translation Engine ingestion, extraction, and export services in Python. Implements Composition Engine runtime and prototype renderer in TypeScript. Maintains shared JSON schema definitions.



UX / Interaction Architect

Translates Shield Runtime Specifications into Figma prototypes and interaction models. Validates that runtime specs produce coherent adaptive experiences. Collaborates with research on human factors testing.



DevOps / Security Engineer

Establishes the CI pipeline, manages secret management infrastructure, enforces security scanning gates, maintains connector isolation environments, and manages deployment configurations.



Technical Writer / Documentation Lead

Maintains engineering documentation in the docs/ directory, produces case study reports, generates executive documentation from GEMS outputs, and ensures Volume IV stays current with implementation.

Acceptance Criteria for MVP

The SHIELD MVP succeeds when a single, documented pipeline run against a real operational source artifact produces all required outputs, passes all validation gates, and results in a human-reviewable prototype that demonstrates the adaptive operational experience encoded in the original source. The following acceptance criteria define the MVP success condition in concrete, testable terms.



Acceptance Criteria

All must be satisfied to declare MVP complete

1 Ingest a real workflow artifact

The pipeline must successfully ingest the SmartTag Appenate JSON definition without data loss, producing a schema-valid corpus output.

2 Preserve provenance

Every generated object must carry complete provenance metadata linking back to the original source artifact through an unbroken chain.

3 Extract entities, events, rules, evidence, connectors

The Operational Knowledge Extractor must produce a complete, schema-valid OKM containing all identified operational objects with confidence scores.

4 Generate OKM, OKG, and OCP

All three modeling outputs must be produced, schema-validated, and committed to the case study directory with their validation metadata.

5 Generate Shield Runtime Specification

A valid Shield Runtime Specification must be produced that defines a complete adaptive interface configuration for the SmartTag operational context.

6 Generate prototype prompt and pass validation

A Gemini Canvas prototype prompt must be generated from the runtime spec and must produce a renderable prototype output when submitted to the target tool.

- Security risks are identified and reported with remediation recommendations
- Human review workflow is initiated and documented for all flagged items
- All outputs are committed to the regression suite for future pipeline validation

Non-Goals

Explicitly defining what the MVP will not do is as important as defining what it will do. Non-goals prevent scope expansion before the core pipeline is validated, protect the evidence-first engineering discipline, and ensure that the first engineering program produces real architectural proof rather than a broad but shallow platform demonstration.

The MVP Will Not

- Build the full LumaShield consumer application
- Launch the marketplace or distribution layer
- Replace existing enterprise systems
- Automate legal or medical judgment
- Train a new foundation model
- Support every connector type at launch
- Eliminate human review from high-consequence workflows
- Produce production-ready mobile applications
- Commercialize every Shield type

Why These Boundaries Exist

Each non-goal is a deliberate scope constraint derived from the evidence-before-expansion principle. The marketplace requires a validated translation pipeline before it has operational assets to distribute. Legal and medical automation requires regulatory approval frameworks that are outside the engineering MVP scope. Foundation model training requires a dataset that the engineering program will create – after the pipeline is proven. Mobile production applications require validated runtime specifications – which the MVP will produce for the first time.

These boundaries are not permanent limitations of the SHIELD architecture. They are sequencing constraints for the first engineering program. Each non-goal becomes a future goal once the MVP evidence supports it.

Research Alignment

The SHIELD engineering program is not isolated from research — it is the evidence engine that makes SHIELD research possible. The MVP pipeline produces structured, versioned, schema-governed outputs that can serve as research artifacts for human coordination studies, cognitive load research, translation science, operational knowledge modeling, and adaptive interaction testing. Engineering and research are co-dependent programs, not separate tracks.



Human Coordination Research

OKG outputs provide structured representations of how humans coordinate around operational events — a direct research artifact for human coordination studies



Cognitive Load Studies

Shield Runtime Specifications define minimal-capability interfaces — prototype outputs enable controlled cognitive load experiments comparing adaptive versus traditional interfaces



Translation Science

The Translation Engine pipeline itself is a research subject — how effectively does structured extraction capture operational meaning from diverse source artifacts?



University Collaboration

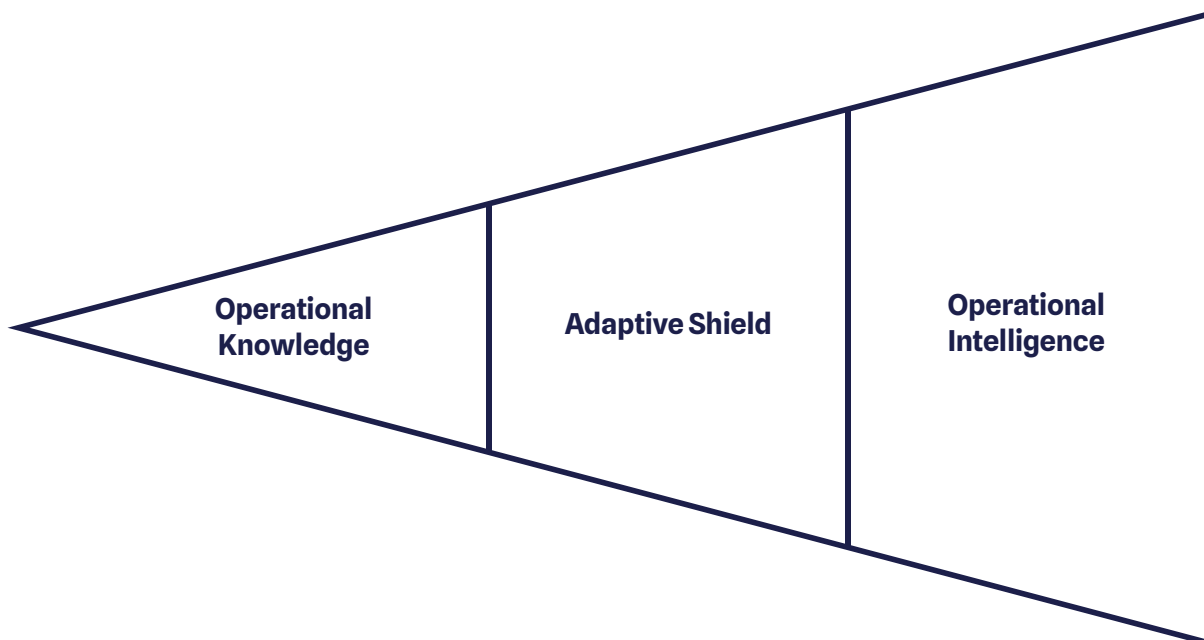
Engineering outputs — corpus entries, extraction reports, OKMs, graphs — are formatted to support university research partner access and replication studies

The engineering MVP creates evidence. Research evaluates the evidence. The feedback loop between engineering outputs and research findings is what allows the SHIELD architecture to evolve on an empirical basis rather than on assumption. University research packages — curated sets of engineering outputs formatted for academic use — are a committed deliverable of the 90-day engineering plan.

Conclusion

Volume IV turns SHIELD from architecture into an engineering program. The three prior volumes established what SHIELD is – its principles, its operational architecture, its human interaction doctrine. Volume IV defines how it begins to be built. The engineering program starts not with the full system, but with the smallest working pipeline that proves the central claim of the architecture: that operational knowledge embedded in real enterprise artifacts can be systematically translated, modeled, composed, and rendered as an adaptive operational experience.

The evidence threshold for the MVP is concrete and testable. A real workflow artifact – SmartTag Irving Oil – enters the pipeline. A validated Shield Runtime Specification and a working prototype emerge from the other end. The knowledge that was implicit in the original workflow is now explicit, structured, version-controlled, governed, and composable. The architecture is no longer a claim. It is a demonstrated result.



SHIELD becomes real when the first operational artifact translates into a working adaptive experience. Volume IV defines every engineering step required to make that moment happen – precisely, practically, and with evidence at every stage.

Terminology and Canonical Reference

The following terminology is canonical across all volumes of the SHIELD Architecture Series. All engineering implementations, schemas, documentation, and communications must use these terms precisely and consistently. No substitutions, renames, or abbreviations are permitted in engineering artifacts without explicit revision to the canon.

Canonical Term	Domain	First Defined
SHIELD	System identity	Volume I
HAIL	Interaction grammar	Volume III
Translation Engine	Architecture component	Volume II
Composition Engine	Architecture component	Volume II
Operational Knowledge Model	Data model	Volume II
Operational Knowledge Graph	Data model	Volume II
Operational Context Model	Data model	Volume II
Operational Context Package	Data model	Volume II
Shield Runtime Specification	Output format	Volume II
Adaptive Operational Workspace	Experience model	Volume III
GEMS	Engineering tool	Volume IV
Shield Integrity Layer	Architecture component	Volume II
Operational Self-Healing Layer	Architecture component	Volume II
Luma ID	Identity system	Volume I
Shield ID	Identity system	Volume I



This document is internally consistent with Volumes I, II, and III of the SHIELD Architecture Series. Any apparent inconsistency between Volume IV and prior volumes should be escalated to the Founder / Product Architect for canon resolution before implementation proceeds.